

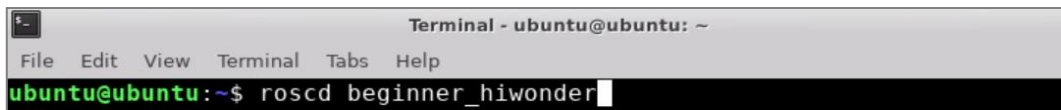
Lesson 7 The Definition and Use Of Topics Message

1. Customize Topic Message

Note: Before customizing the topic message, the workspace and package need to be created first. The specific operation steps can be viewed in “ROS Basic Lessons/Lesson 3 Create Workspace and Package.”

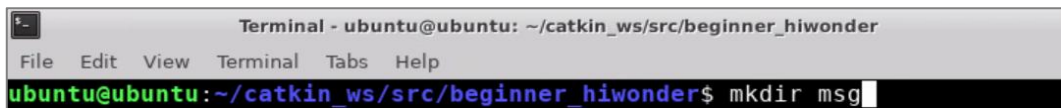
The specific operation steps for customizing topic messages are as following:

- 1) Open the terminal.
- 2) Enter “roscd beginner_hiwonder” command to locate to the package directory and press “Enter”.

A terminal window titled "Terminal - ubuntu@ubuntu: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The command prompt shows "ubuntu@ubuntu: ~\$ roscd beginner_hiwonder" with a cursor at the end.

Note: If there is a prompt “No such package/stack ‘beginner_hiwonder’” appears, it means that the package does not exist in the environment variable ROS_PACKAGE_PATH. The specific solution can be viewed in “ROS Basic Lessons/Lesson 3 Create Workspace and Package”. After the problem is solved, please repeat the current step.

- 3) Enter “mkdir msg” command and press “Enter”. Then create a new folder “msg” for storing text files.

A terminal window titled "Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The command prompt shows "ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder\$ mkdir msg" with a cursor at the end.

- 4) Enter “cd msg” command and press “Enter”.

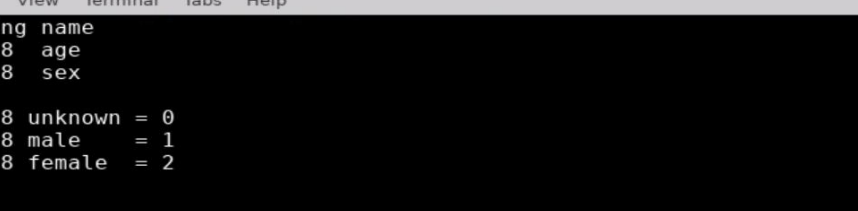
```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ cd msg
```

5) Enter “vi Person.msg” command to edit program and copy the following program. If want to modify, you can press “i” key. After modifying, press “Esc” and enter “:wq” to save and exit.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/msg
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/msg$ vi Person.msg
```

```
string name
uint8 age
uint8 sex

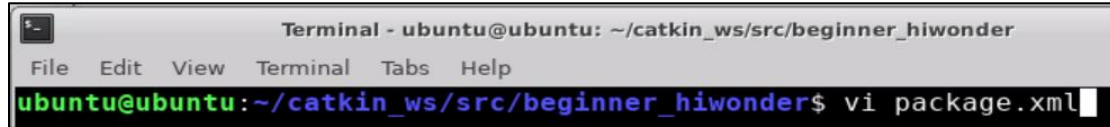
uint8 unknown = 0
uint8 male = 1
uint8 female = 2
```



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/msg
File Edit View Terminal Tabs Help
1 string name
2 uint8 age
3 uint8 sex
4
5 uint8 unknown = 0
6 uint8 male = 1
7 uint8 female = 2
: wq
```

6) Enter “vi package.xml” command. Then copy the following program and

add the package dependencies in the position shown in the figure below. If want to modify, you can press “i” again. After modifying, press “Esc” and enter “:wq” to save and exit.

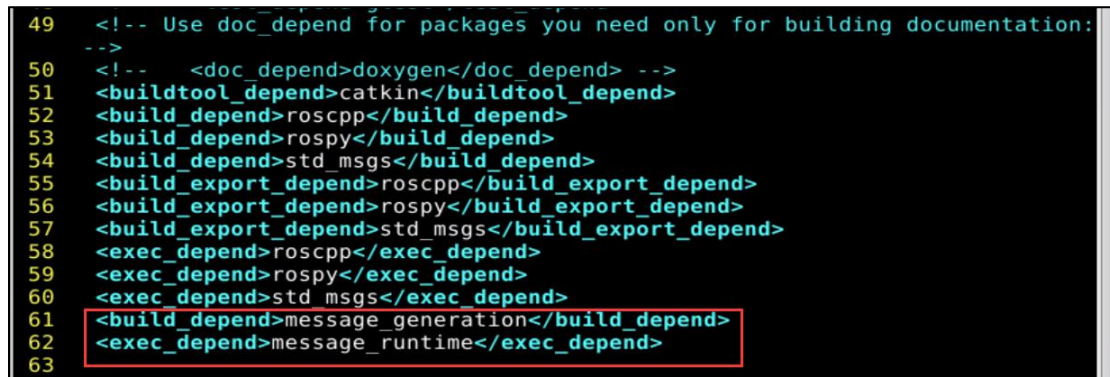


```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ vi package.xml
```

7) Add the package dependencies in the position shown in the below figure:

```
<build_depend>message_generation</build_depend>

<exec_depend>message_runtime</exec_depend>
```



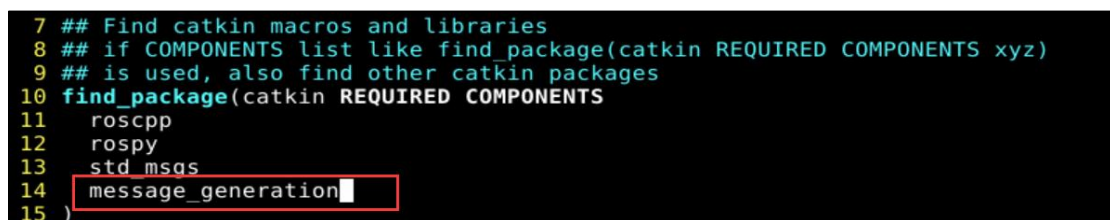
```
49 <!-- Use doc_depend for packages you need only for building documentation:
-->
50 <!-- <doc_depend>doxygen</doc_depend> -->
51 <buildtool_depend>catkin</buildtool_depend>
52 <build_depend>roscpp</build_depend>
53 <build_depend>rospy</build_depend>
54 <build_depend>std_msgs</build_depend>
55 <build_export_depend>roscpp</build_export_depend>
56 <build_export_depend>rospy</build_export_depend>
57 <build_export_depend>std_msgs</build_export_depend>
58 <exec_depend>roscpp</exec_depend>
59 <exec_depend>rospy</exec_depend>
60 <exec_depend>std_msgs</exec_depend>
61 <build_depend>message_generation</build_depend>
62 <exec_depend>message_runtime</exec_depend>
63
```

8) Enter “vi CMakeLists.txt” and press “i” to modify “CMakeLists.txt” file.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ vi CMakeLists.txt
```

9) Add the required compilation option “message_generation” in the position shown in the figure below.



```
7 ## Find catkin macros and libraries
8 ## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
9 ## is used, also find other catkin packages
10 find_package(catkin REQUIRED COMPONENTS
11   roscpp
12   rospy
13   std_msgs
14   message_generation
15 )
```

10) Locate the code shown in the figure below. Then uncomment the framed code and add the required compilation option “Person.msg”.

```

49
50 ## Generate messages in the 'msg' folder
51 # add_message_files(
52 #     FILES
53 #     Message1.msg
54 #     Message2.msg
55 # )
56

```

```

49
50 ## Generate messages in the 'msg' folder
51 add_message_files(
52     FILES
53     Person.msg
54 )
55

```

11) Find the code shown in the figure below. Then uncomment the code in red box and ensure that the required compilation options take effect.

```

69
70 ## Generate added messages and services with any dependencies listed here
71 # generate_messages(
72 #     DEPENDENCIES
73 #     std_msgs
74 # )
75

```

```

69
70 ## Generate added messages and services with any dependencies listed here
71 generate_messages(
72     DEPENDENCIES
73     std_msgs
74 )
75

```

12) Find the code shown in the figure below. Then uncomment the code in red box and add the required compilation option “message_runtime”.

```

105 catkin_package(
106 #   INCLUDE_DIRS include
107 #   LIBRARIES beginner hiwonder
108 #   CATKIN_DEPENDS roscpp rospy std_msgs
109 #   DEPENDS system_lib
110 )

```

```

105 catkin_package(
106 #   INCLUDE_DIRS include
107 #   LIBRARIES beginner hiwonder
108 CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
109 #   DEPENDS system_lib
110 )
111

```

13) After modifying, press “Esc” and enter “:wq” to save and exit.

```

184 # FILES_MATCHING PATTERN "*.h"
185 # PATTERN ".svn" EXCLUDE
186 # )
187
188 ## Mark other files for installation (e.g. launch and bag files, etc.)
189 # install(FILES
190 #   myfile1
191 #   myfile2
192 #   DESTINATION ${CATKIN_PACKAGE_SHARE_DESTINATION}
193 # )
194
195 #####
196 ## Testing ##
197 #####
198
199 ## Add gtest based cpp test target and link libraries
200 # catkin_add_gtest(${PROJECT_NAME}-test test/test_beginner_hiwonder.cpp)
201 # if(TARGET ${PROJECT_NAME}-test)
202 #   target_link_libraries(${PROJECT_NAME}-test ${PROJECT_NAME})
203 # endif()
204
205 ## Add folders to be run by python nosetests
206 # catkin_add_nosetests(test)
:wg

```

14) Enter the command “rosmmsg show beginner_hiwonder/Person” and press “Enter” to check whether the message written can be recognized by system. When the words shown in red box appear, it means that they are recognized successfully.

```

ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ rosmmsg show beginner_hiwonder/Person
uint8 unknown=0
uint8 male=1
uint8 female=2
string name
uint8 age
uint8 sex

```

2. The Use of Topic Message

2.1 Create Publisher and Subscriber Code

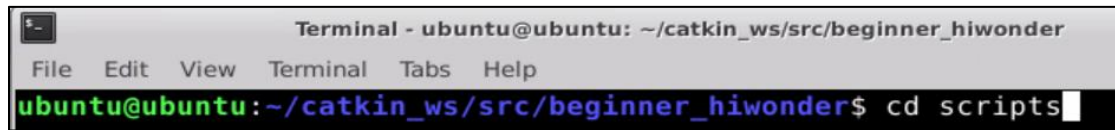
- 1) Open the terminal.
- 2) Enter “roscd beginner_hiwonder” command to locate to the package directory and press “Enter”.

```

Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ roscd beginner_hiwonder

```

3) Enter “cd scripts” command and press “Enter” to come to the folder “scripts” where Python scripts are stored.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ cd scripts
```

4) Enter “vi person_publisher.py” command to edit program, and then copy the following program. If want to modify, you can press “i”. After modifying, press “Esc” and enter “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi person_publisher.py
```

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

#This routine will publish /person_info topic and customize message type
beginner_hiwonder::Person.

import rospy

from beginner_hiwonder.msg import Person

def velocity_publisher():

    # Initialize ROS node

    rospy.init_node('person_publisher', anonymous=True)

    # Create a publisher and publish a topic named /person_info. The message type is
beginner_hiwonder::Person and the queue length is 10.
```

```
person_info_pub = rospy.Publisher('/person_info', Person, queue_size=10)

#set the loop rate

rate = rospy.Rate(10)

while not rospy.is_shutdown():

    # Initialize the message of beginner_hiwonder::Person type

    person_msg = Person()

    person_msg.name = "Tom";

    person_msg.age  = 18;

    person_msg.sex  = Person.male;

    # publish message

    person_info_pub.publish(person_msg)

    rospy.loginfo("Publsh person message[%s, %d, %d]",

                  person_msg.name, person_msg.age, person_msg.sex)

    # Delay on basis of the loop rate

    rate.sleep()

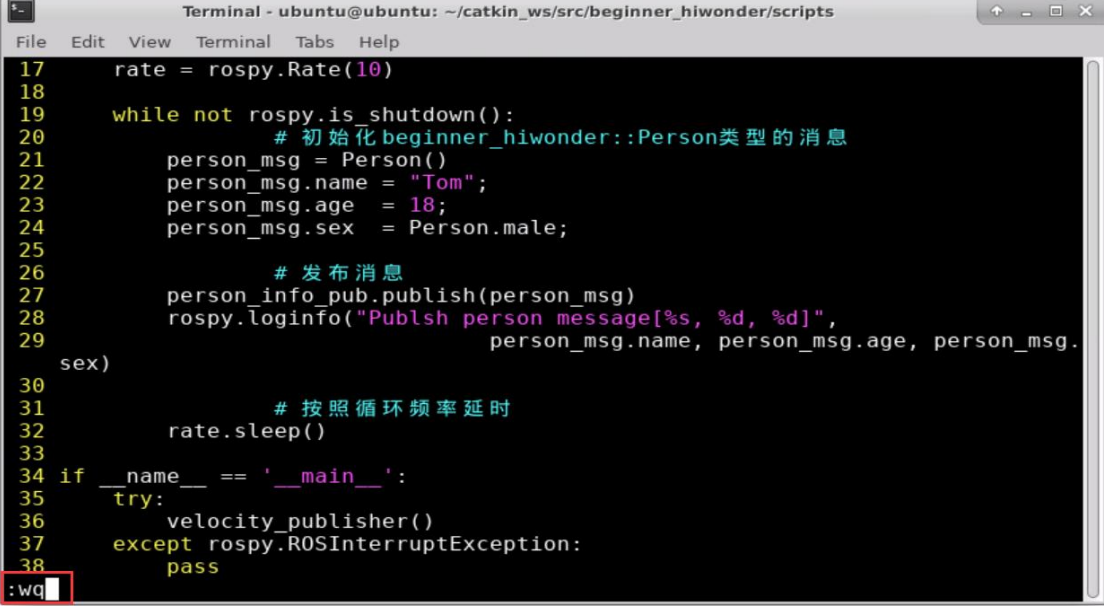
if __name__ == '__main__':

    try:
```

```
velocity_publisher()

except rospy.ROSInterruptException:

    pass
```



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
17 rate = rospy.Rate(10)
18
19 while not rospy.is_shutdown():
20     # 初始化beginner_hiwonder::Person类型的消息
21     person_msg = Person()
22     person_msg.name = "Tom";
23     person_msg.age = 18;
24     person_msg.sex = Person.male;
25
26     # 发布消息
27     person_info_pub.publish(person_msg)
28     rospy.loginfo("Publish person message[%s, %d, %d]",
29                 person_msg.name, person_msg.age, person_msg.
sex)
30
31     # 按照循环频率延时
32     rate.sleep()
33
34 if __name__ == '__main__':
35     try:
36         velocity_publisher()
37     except rospy.ROSInterruptException:
38         pass
:wq
```

5) Enter “vi person_subscriber.py” to edit program, and copy the following program. If want to modify, you can press “i”. After modifying, press “Esc” and enter “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi person_subscriber.py
```

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

# This routine will subscribe to the /person_info topic and customize the message type
beginner_hiwonder::Person

import rospy
```

```
from beginner_hiwonder.msg import Person

def personInfoCallback(msg):

    rospy.loginfo("Subscribe Person Info: name:%s  age:%d  sex:%d",

                  msg.name, msg.age, msg.sex)

def person_subscriber():

    # Initialize ROS node

    rospy.init_node('person_subscriber', anonymous=True)

    # Create a subscriber, subscribe the topic named /person_info and register the callback
funcation personInfoCallback

    rospy.Subscriber("/person_info", Person, personInfoCallback)

    # loop and wait the callback function

    rospy.spin()

if __name__ == '__main__':

    person_subscriber()
```

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 # 该例程将订阅/person_info话题，自定义消息类型beginner_hiwonder::Person
5
6 import rospy
7 from beginner_hiwonder.msg import Person
8
9 def personInfoCallback(msg):
10     rospy.loginfo("Subscribe Person Info: name:%s age:%d sex:%d",
11                 msg.name, msg.age, msg.sex)
12
13 def person_subscriber():
14     # ROS节点初始化
15     rospy.init_node('person_subscriber', anonymous=True)
16
17     # 创建一个Subscriber，订阅名为/person_info的topic，注册回调函数personInfoCallback
18     rospy.Subscriber("/person_info", Person, personInfoCallback)
19
20     # 循环等待回调函数
21     rospy.spin()
22
23 :wq
```

6) Enter “chmod +x person_publisher.py” command and press “Enter” to give the executable permission to the saved person_publisher.py.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x person_publisher.py
```

2.2 Run Publisher And Subscriber Nodes

1) Enter “cd ~/catkin_ws” command and press “Enter” to enter to catkin workspace.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ cd ~/catkin_ws
```

2) Enter “catkin_make” command and press “Enter” to build all the packages in directory.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ catkin_make
```

3) Enter “source ./devel/setup.bash” command and press “Enter” to refresh the workspace environment.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ source ./devel/setup.bash
```

- 4) Enter “roscore” command to start node manager.

```
ubuntu@ubuntu:~/catkin_ws$ roscore
```

After starting, the prompt below will appear:

```
RLException: roscore cannot run as another roscore/master is already running.
Please kill other roscore/master processes before relaunching.
The ROS_MASTER_URI is http://ubuntu:11311/
The traceback for the exception was written to the log file
ubuntu@ubuntu:~/catkin_ws$
```

- 5) Enter “roslaunch beginner_hiwonder person_publisher.py” command and press “Enter” to run publisher node. If want to stop running node, you can press “Ctrl+C”.

```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ roslaunch beginner_hiwonder person_publisher.py
[INFO] [1644662104.056406]: Pubsh person message[Tom, 18, 1]
[INFO] [1644662104.157919]: Pubsh person message[Tom, 18, 1]
[INFO] [1644662104.256514]: Pubsh person message[Tom, 18, 1]
[INFO] [1644662104.356546]: Pubsh person message[Tom, 18, 1]
[INFO] [1644662104.456802]: Pubsh person message[Tom, 18, 1]
```

- 6) Open a new terminal. Enter “roslaunch beginner_hiwonder person_subscriber.py” command and press “Enter” to run the subscriber node. If want to stop running node, you can press “Ctrl+C”.

```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ roslaunch beginner_hiwonder person_subscriber.py
[INFO] [1644662179.557655]: Subscribe Person Info: name:Tom age:18 sex:1
[INFO] [1644662179.657228]: Subscribe Person Info: name:Tom age:18 sex:1
[INFO] [1644662179.758308]: Subscribe Person Info: name:Tom age:18 sex:1
[INFO] [1644662179.858379]: Subscribe Person Info: name:Tom age:18 sex:1
[INFO] [1644662179.957714]: Subscribe Person Info: name:Tom age:18 sex:1
```

① The publisher node needs to be started first, and then the subscriber node can subscribe message.

② If need to receive the publisher messages completely, you can start the subscriber node first and then the publisher node.

Lesson 8 Write A Simple Client

This section takes the creation of a simple service (Client) node `turtle_spawn.py` as an example to explain and this node publishes a request for the client to spawn a new turtle by means of a program.

1. Configure Client Code Compilation Rule

- 1) Enter “`cd catkin_ws/src/beginner_hiwonder/scripts/`” command and press “Enter”.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ cd catkin_ws/src/beginner_hiwonder/scripts/
```

- 2) Enter “`vi turtle_spawn.py`” command to edit program and copy the following program. If want to program, you can press “i”, and then press “Esc” to enter “`:wq`” to exit and save.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi turtle_spawn.py
```

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-


# This routine will request /spawn service and the service data type is
turtlesim::Spawn


import sys

import rospy
```

```
from turtlesim.srv import Spawn

def turtle_spawn():

    # Initialize ROS node

    rospy.init_node('turtle_spawn')

    # After finding the /spawn service, create a service client, and then connect the
service named /spawn.

    rospy.wait_for_service('/spawn')

    try:

        add_turtle = rospy.ServiceProxy('/spawn', Spawn)

        # Request service call and input request data

        response = add_turtle(2.0, 2.0, 0.0, "turtle2")

        return response.name

    except rospy.ServiceException, e:

        print "Service call failed: %s"%e

if __name__ == "__main__":

    #The service calls and displays the result of call.

    print "Spwan turtle successfully [name:%s]" %(turtle_spawn())
```

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
6
7 import sys
8 import rospy
9 from turtlesim.srv import Spawn
10
11 def turtle_spawn():
12     # ROS节点初始化
13     rospy.init_node('turtle_spawn')
14
15     # 发现 /spawn服务后，创建一个服务客户端，连接名为 /spawn的 service
16     rospy.wait_for_service('/spawn')
17     try:
18         add_turtle = rospy.ServiceProxy('/spawn', Spawn)
19
20         # 请求服务调用，输入请求数据
21         response = add_turtle(2.0, 2.0, 0.0, "turtle2")
22         return response.name
23     except rospy.ServiceException, e:
24         print "Service call failed: %s"%e
25
26 if __name__ == "__main__":
27     #服务调用并显示调用结果
28     print "Spwan turtle successfully [name:%s]" %(turtle_spawn())
:wq

```

- 3) Enter “chmod +x turtle_spawn.py” command and press “Enter” to give the executable permission to the saved turtle_spawn.py.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x turtle_spawn.py

```

2. Run Client

- 1) Enter “roscore” command to start node manager.

```

ubuntu@ubuntu:~/catkin_ws$ roscore

```

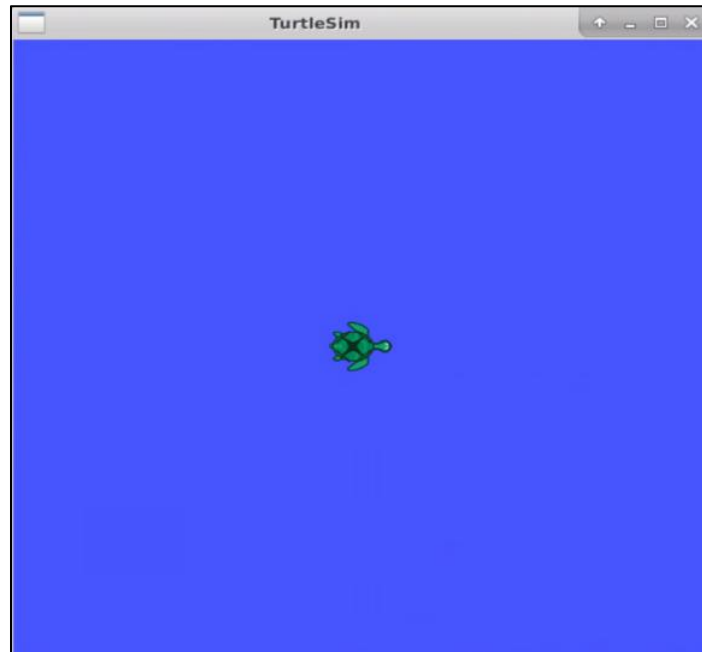
- 2) Enter “roslaunch turtlesim turtlesim_node” command and press “enter” to run turtlesim.

```

ubuntu@ubuntu:~/catkin_ws$ roslaunch turtlesim turtlesim_node

```

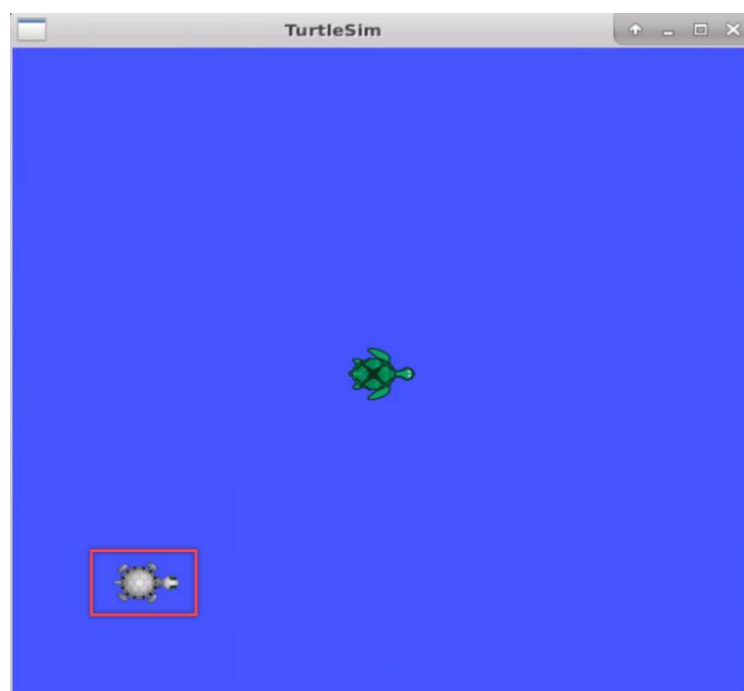
At this time, the interface will pop up the turtlesim window, as the figure shown below:



- 3) Open a new terminal. Enter “roslaunch beginner_hiwonder turtle_spawn.py” command and press “Enter” to run the client.

```
ubuntu@ubuntu:~$ roslaunch beginner_hiwonder turtle_spawn.py
Spawn turtle successfully [name:turtle2]
ubuntu@ubuntu:~$
```

At this time, client will send the request to server and respond to start another turtle.

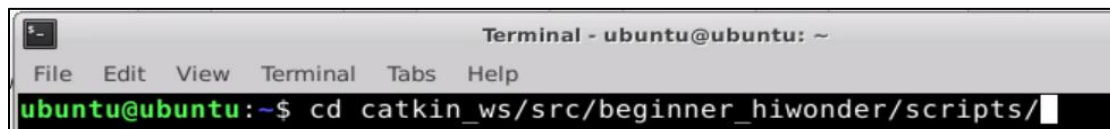


Lesson 9 Write A Simple Server

1. Create Service Code

Note: Before creating service code, you need to create workspace and package first. The specific operation steps can be viewed in file "ROS Basic Lessons->Lesson 3 Create Workspace and Package".

- 1) Enter "`cd catkin_ws/src/beginner_hiwonder/scripts/`" command and press "Enter".



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ cd catkin_ws/src/beginner_hiwonder/scripts/
```

- 2) Enter "`vi turtle_command_server.py`" command to edit program and copy the following program. If want to modify, you can press "i". After modifying, press "Esc" and enter ":wq" to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi turtle_command_server.py
```

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

import rospy

import thread,time

from geometry_msgs.msg import Twist

from std_srvs.srv import Trigger, TriggerResponse

pubCommand = False;

turtle_vel_pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)

def command_thread():
```

```
while True:

    if pubCommand:

        vel_msg = Twist()

        vel_msg.linear.x = 0.5

        vel_msg.angular.z = 0.2

        turtle_vel_pub.publish(vel_msg)

    time.sleep(0.1)

def commandCallback(req):

    global pubCommand

    pubCommand = bool(1-pubCommand)

    rospy.loginfo("Publish turtle velocity command![%d]", pubCommand)

    return TriggerResponse(1, "Change turtle command state!")

def turtle_command_server():

    rospy.init_node('turtle_command_server')

    s = rospy.Service('/turtle_command', Trigger, commandCallback)

    print "Ready to receive turtle command."

    thread.start_new_thread(command_thread, ())

    rospy.spin()

if __name__ == "__main__":

    turtle_command_server()
```

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
8 turtle_vel_pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
9 def command_thread():
10     while True:
11         if pubCommand:
12             vel_msg = Twist()
13             vel_msg.linear.x = 0.5
14             vel_msg.angular.z = 0.2
15             turtle_vel_pub.publish(vel_msg)
16
17             time.sleep(0.1)
18 def commandCallback(req):
19     global pubCommand
20     pubCommand = bool(1-pubCommand)
21     rospy.loginfo("Publish turtle velocity command![%d]", pubCommand)
22     return TriggerResponse(1, "Change turtle command state!")
23 def turtle_command_server():
24     rospy.init_node('turtle_command_server')
25     s = rospy.Service('/turtle_command', Trigger, commandCallback)
26     print "Ready to receive turtle command."
27     thread.start_new_thread(command_thread, ())
28     rospy.spin()
29 if __name__ == "__main__":
30     turtle_command_server()
:wq

```

- 3) Enter “**chmod +x turtle_command_server.py**” command and press “Enter” to give the executable permission to the saved turtle_command_server.py.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x turtle_command
_server.py

```

2. Run Server Node

- 1) Enter “roscore” command to start node manager.

```

ubuntu@ubuntu:~/catkin_ws$ roscore

```

- 2) Enter “roslaunch turtlesim turtlesim_node” command and press “Enter” to start turtlesim simulator window.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ roslaunch turtlesim turtlesim_node

```

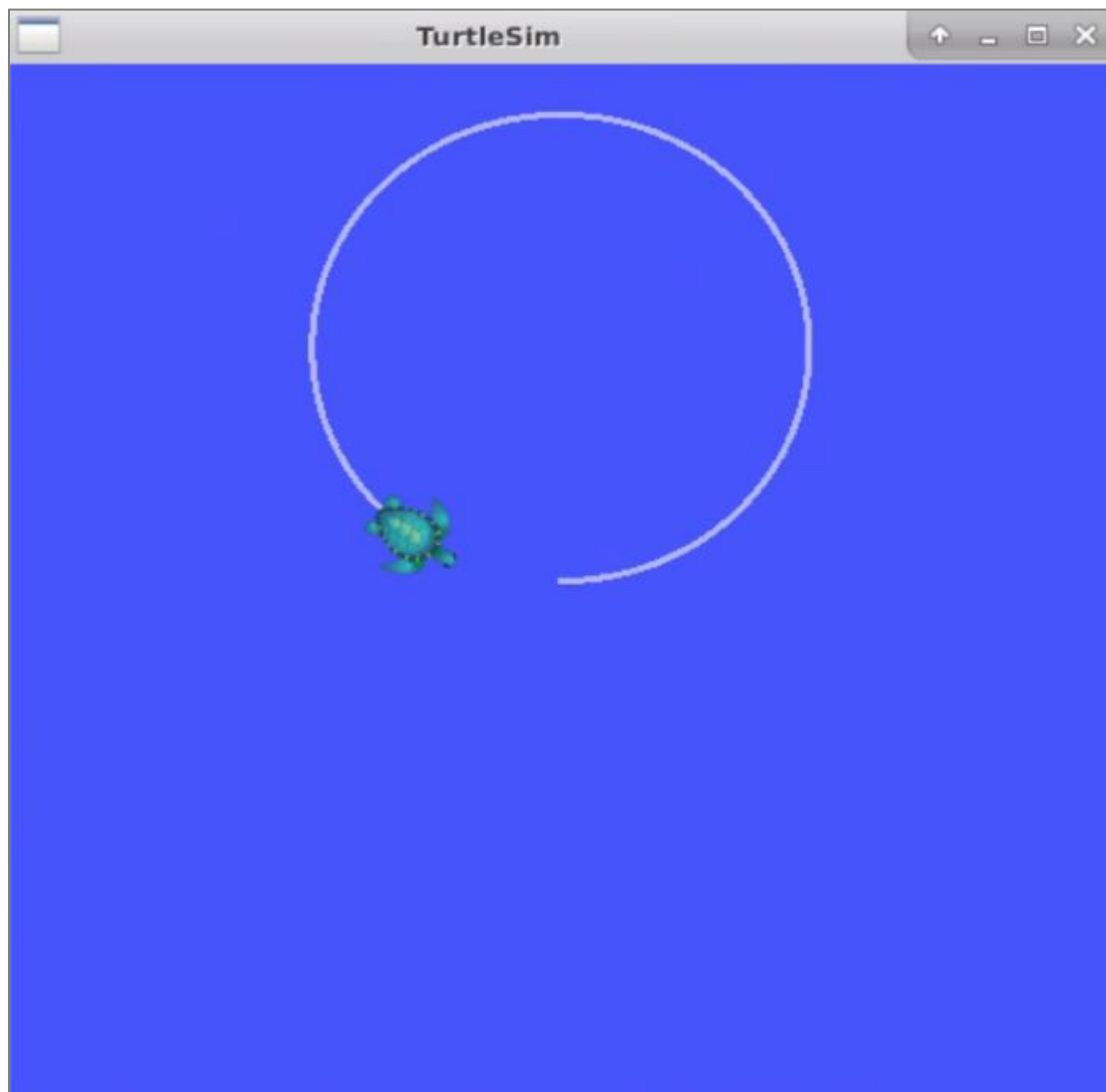
- 3) Open a new terminal. Then enter “roslaunch beginner_hiwonder turtle_command_server.py” and press “Enter” to run service node. If need

to stop running the node, you can press “Ctrl+C”.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosrn beginner_hiwonder turtle_command_server.py
```

- 4) Open a new terminal again. Enter “rosservice call /turtle_command “{}”” command and press “Enter” to move the turtle along the circular path.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosservice call /turtle_command "{}"  
success: True  
message: "Change turtle command state!"
```



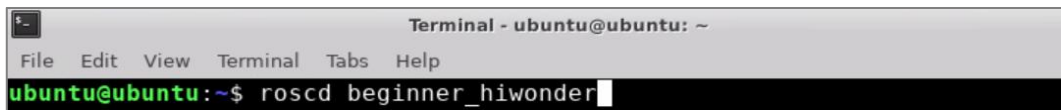
Lesson 10 The Definition and Use of Service Data

1. Customize Service Data

Note: Before customizing service data, the workspace and package need to be created first. The specific operation steps can be viewed in file “ROS Basic Lessons/Lesson 3 Create Workspace and Package”.

The specific operation steps to customize service data are as follow:

- 1) Open the terminal.
- 2) Enter “`roscd beginner_hiwonder`” command to go to the package directory and press “Enter”.



Note: If there is a prompt “**No such package/stack ‘beginner_hiwonder’**” appears, it means that the package does not exist in the environment variable ROS_PACKAGE_PATH. The specific solution can be viewed in “ROS Basic Lessons/Lesson 3 Create Workspace and Package”. After the problem is solved, please repeat the current step.

- 3) Enter “`mkdir srv`” command and press “Enter”. Then create a new folder “srv” for storing text files.



- 4) Enter “`vi Person.srv`” command to edit program, and then copy the following program. If want to modify, you can press “i”. After modifying, press “Esc” and enter “`:wq`” to save and exit.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/srv
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/srv$ vi Person.srv
```

```
string name

int8 age

int8 sex

int8 unknown = 0

int8 male    = 1

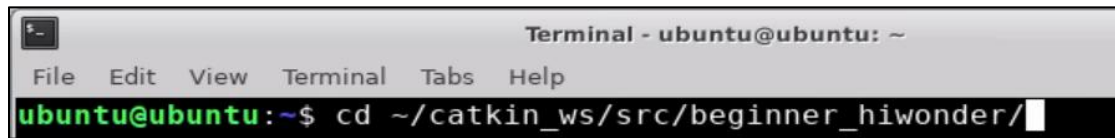
int8 female  = 2

---

string result
```

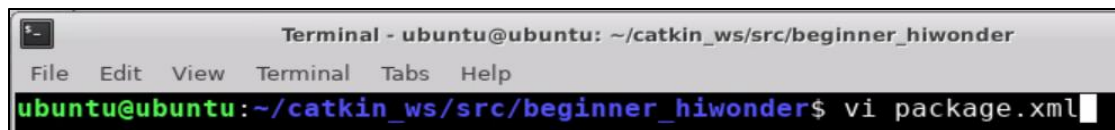
```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/srv
File Edit View Terminal Tabs Help
1 string name
2 int8 age
3 int8 sex
4
5 int8 unknown = 0
6 int8 male    = 1
7 int8 female  = 2
8
9 ---
10 string result
: wq
```

5) Enter “cd ~/catkin_ws/src/beginner_hiwonder/” command, and then press “Enter”.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ cd ~/catkin_ws/src/beginner_hiwonder/
```

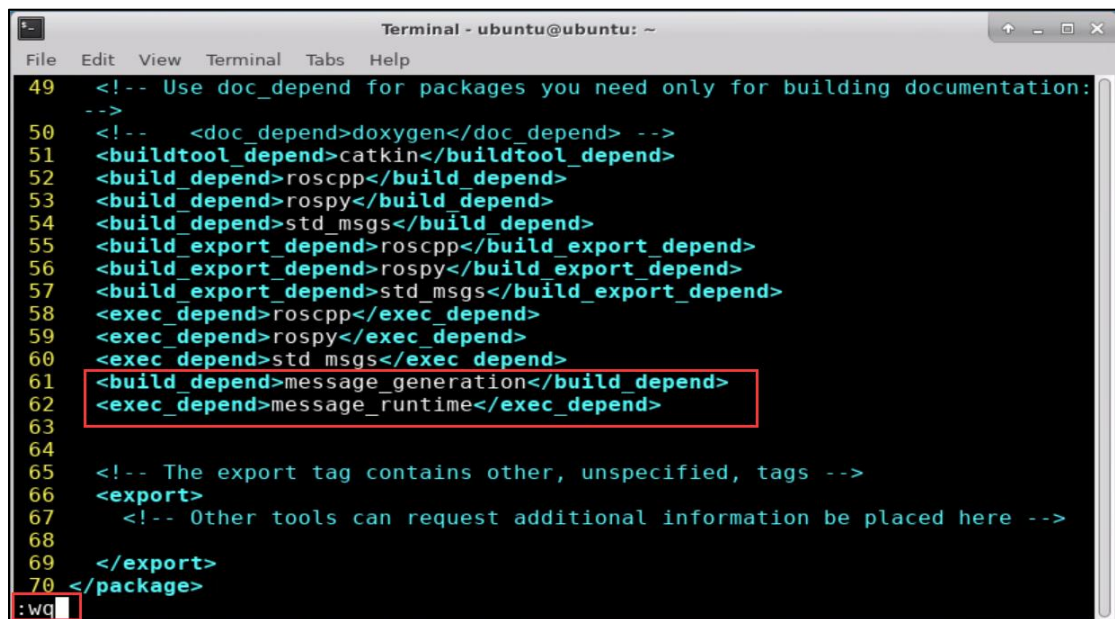
6) Enter “vi package.xml” command. Then copy the following program and add the package dependencies in the position shown in the figure below. If want to modify, you can press “i” again. After modifying, press “Esc” and enter “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ vi package.xml
```

```
<build_depend>message_generation</build_depend>

<exec_depend>message_runtime</exec_depend>
```



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
49 <!-- Use doc_depend for packages you need only for building documentation:
-->
50 <!-- <doc_depend>doxygen</doc_depend> -->
51 <buildtool_depend>catkin</buildtool_depend>
52 <build_depend>roscpp</build_depend>
53 <build_depend>rospy</build_depend>
54 <build_depend>std_msgs</build_depend>
55 <build_export_depend>roscpp</build_export_depend>
56 <build_export_depend>rospy</build_export_depend>
57 <build_export_depend>std_msgs</build_export_depend>
58 <exec_depend>roscpp</exec_depend>
59 <exec_depend>rospy</exec_depend>
60 <exec_depend>std_msgs</exec_depend>
61 <build_depend>message_generation</build_depend>
62 <exec_depend>message_runtime</exec_depend>
63
64
65 <!-- The export tag contains other, unspecified, tags -->
66 <export>
67 <!-- Other tools can request additional information be placed here -->
68
69 </export>
70 </package>
:wq
```

7) Enter “vi CMakeLists.txt” and press “i” to modify “CMakeLists.txt” file.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ vi CMakeLists.txt
```

8) Add the required compilation option “message_generation” in the position shown in the figure below.

```

6
7 ## Find catkin macros and libraries
8 ## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
9 ## is used, also find other catkin packages
10 find_package(catkin REQUIRED COMPONENTS
11   roscpp
12   rospy
13   std_msgs
14   message_generation
15 )

```

9) Find the code shown in the figure below. Then uncomment the framed code and add the required compilation option “Person.srv”.

```

53 #   Message1.msg
54 #   Message2.msg
55 # )
56
57 ## Generate services in the 'srv' folder
58 add_service_files(
59   FILES
60   Person.srv
61 )
62
63 ## Generate actions in the 'action' folder
64 # add_action_files(

```

10) Find the code shown in the figure below. Then uncomment the code in red box and ensure that the required compilation options take effect.

```

67 #   Action2.action
68 # )
69
70 ## Generate added messages and services with any dependencies listed here
71 generate_messages(
72   DEPENDENCIES
73   std_msgs
74 )
75
76 #####
77 ## Declare ROS dynamic reconfigure parameters ##
78 #####

```

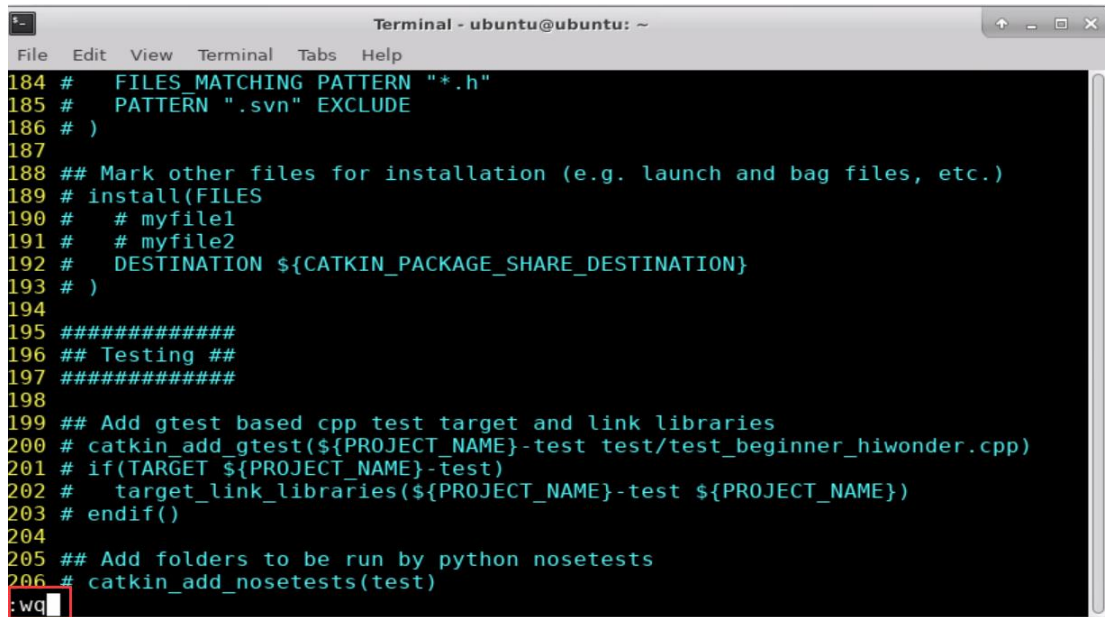
11) Find the code shown in the figure below. Then uncomment the code in red box and add the required compilation option “message_runtime”.

```

103 ## CATKIN_DEPENDS: catkin_packages dependent projects also need
104 ## DEPENDS: system dependencies of this project that dependent projects also
105   need
106 catkin_package(
107 #   INCLUDE_DIRS include
108 #   LIBRARIES beginner_hiwonder
109   CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
110 #   DEPENDS system_lib
111 )

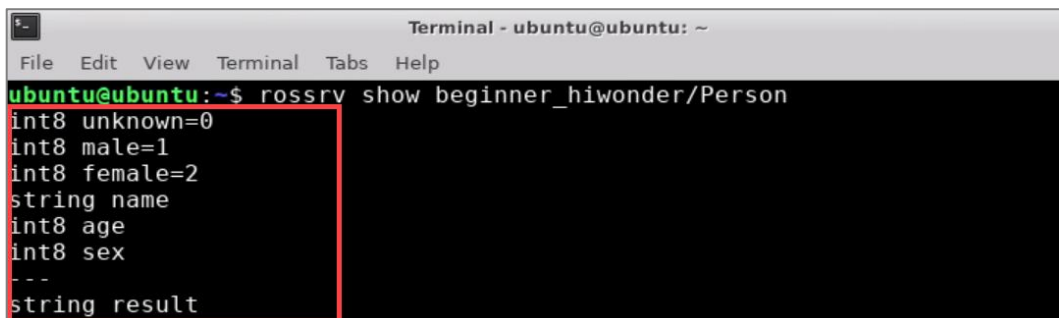
```

12) After modifying, press “Esc” and enter “:wq” to save and exit.

A terminal window titled "Terminal - ubuntu@ubuntu: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal displays the content of a CMakeLists.txt file, with line numbers 184 through 206 on the left. The code includes file matching patterns, installation instructions, testing markers, and catkin test target definitions. At the bottom, the prompt "ubuntu@ubuntu:~\$ " is followed by the text "wd" which is enclosed in a red rectangular box.

```
184 # FILES_MATCHING PATTERN "*.h"
185 # PATTERN ".svn" EXCLUDE
186 # )
187
188 ## Mark other files for installation (e.g. launch and bag files, etc.)
189 # install(FILES
190 #   # myfile1
191 #   # myfile2
192 #   DESTINATION ${CATKIN_PACKAGE_SHARE_DESTINATION}
193 # )
194
195 #####
196 ## Testing ##
197 #####
198
199 ## Add gtest based cpp test target and link libraries
200 # catkin_add_gtest(${PROJECT_NAME}-test test/test_beginner_hiwonder.cpp)
201 # if(TARGET ${PROJECT_NAME}-test)
202 #   target_link_libraries(${PROJECT_NAME}-test ${PROJECT_NAME})
203 # endif()
204
205 ## Add folders to be run by python nosetests
206 # catkin_add_nosetests(test)
ubuntu@ubuntu:~$ wd
```

13) Enter the command “rosmv show beginner_hiwonder/Person” and press “Enter” to check whether the message written can be recognized by system. When the words shown in red box appear, it means that they are recognized successfully.

A terminal window titled "Terminal - ubuntu@ubuntu: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows the command "rosmv show beginner_hiwonder/Person" being executed. The output lists the fields of the Person message type: int8 unknown=0, int8 male=1, int8 female=2, string name, int8 age, int8 sex, and a separator ---. The final line is "string result". The output from "int8 unknown=0" down to "string result" is enclosed in a red rectangular box.

```
ubuntu@ubuntu:~$ rosmv show beginner_hiwonder/Person
int8 unknown=0
int8 male=1
int8 female=2
string name
int8 age
int8 sex
---
string result
```

2. The Use of Service Data

2.1 Create Server and Client Data

- 1) Open the terminal.
- 2) Enter “cd catkin_ws/src/beginner_hiwonder/scripts/” command and press “Enter” to come to the folder “scripts” where Python scripts are stored.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ cd catkin_ws/src/beginner_hiwonder/scripts/
```

- 3) Enter “vi person_server.py” command to edit program, and then copy the following program. If want to modify, you can press “i”. After modifying, press “Esc” and enter “:wq” to save and exit.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi person_server.py
```

```
#!/usr/bin/env python  
  
# -*- coding: utf-8 -*-  
  
# This routine will execute the /show_perso service with service data type  
beginner_hiwonder::Person  
  
import rospy  
  
from beginner_hiwonder.srv import Person, PersonResponse  
  
def personCallback(req):  
  
    # Display request message  
  
    rospy.loginfo("Person: name:%s age:%d sex:%d", req.name, req.age, req.sex)  
  
    # Data feedback  
  
    return PersonResponse("OK")
```

```
def person_server():

    # Initialize ROS node

    rospy.init_node('person_server')

    # Create a server named /show_person and register personCallback function

    s = rospy.Service('/show_person', Person, personCallback)

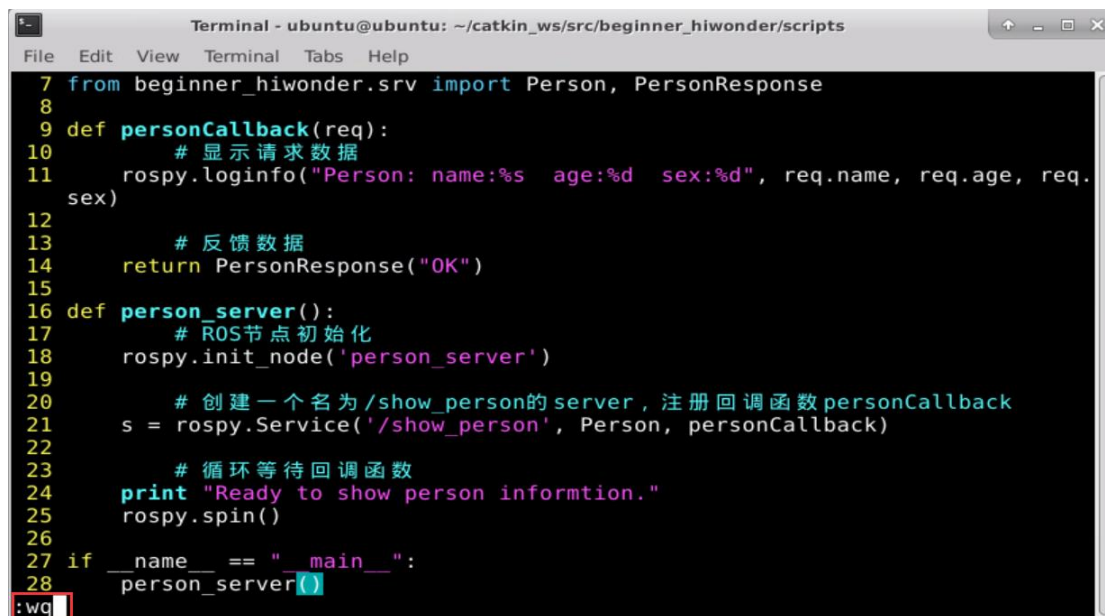
    # loop and wait callback function

    print "Ready to show person informtion."

    rospy.spin()

if __name__ == "__main__":

    person_server()
```



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
7 from beginner_hiwonder.srv import Person, PersonResponse
8
9 def personCallback(req):
10     # 显示请求数据
11     rospy.loginfo("Person: name:%s age:%d sex:%d", req.name, req.age, req.
sex)
12
13     # 反馈数据
14     return PersonResponse("OK")
15
16 def person_server():
17     # ROS节点初始化
18     rospy.init_node('person_server')
19
20     # 创建一个名为/show_person的server, 注册回调函数personCallback
21     s = rospy.Service('/show_person', Person, personCallback)
22
23     # 循环等待回调函数
24     print "Ready to show person informtion."
25     rospy.spin()
26
27 if __name__ == "__main__":
28     person_server()
:wq
```

- 4) Enter “vi person_subscriber.py” to edit program, and copy the following program. If want to modify, you can press “i”. After modifying, press “Esc”

and enter “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi person_client.py
```

```
#!/usr/bin/env python

# -*- coding: utf-8 -*-

# This routine will request /show_person service with the service data type
beginner_hiwonder::Person

import sys

import rospy

from beginner_hiwonder.srv import Person, PersonRequest

def person_client():

    # Initialize ROS node

    rospy.init_node('person_client')

    # After finding the /spawn service, create a service client, and then connect service name
    /spawn

    rospy.wait_for_service('/show_person')

    try:

        person_client = rospy.ServiceProxy('/show_person', Person)

    # Request service call, input request data
```

```

        response = person_client("Tom", 20, PersonRequest.male)

        return response.result

    except rospy.ServiceException, e:

        print "Service call failed: %s"%e

if __name__ == "__main__":

    #Service call and display the call result

    print "Show person result : %s" %(person_client())

```

```

5
6 import sys
7 import rospy
8 from beginner_hiwonder.srv import Person, PersonRequest
9
10 def person_client():
11     # ROS节点初始化
12     rospy.init_node('person_client')
13
14     # 发现 /spawn服务后，创建一个服务客户端，连接名为 /spawn的 service
15     rospy.wait_for_service('/show_person')
16     try:
17         person_client = rospy.ServiceProxy('/show_person', Person)
18
19         # 请求服务调用，输入请求数据
20         response = person_client("Tom", 20, PersonRequest.male)
21         return response.result
22     except rospy.ServiceException, e:
23         print "Service call failed: %s"%e
24
25 if __name__ == "__main__":
26     #服务调用并显示调用结果
27     print "Show person result : %s" %(person_client())
:wg

```

- 5) Enter “chmod +x person_server.py” and “chmod +x person_client.py” command, and then press “Enter” to give the executable permission to the saved person_publisher.py.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x person_server.py

```

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x person_client.py

```

2.2 Run Server and Client Nodes

- 1) Enter “cd ~/catkin_ws” command and press “Enter” to enter to catkin workspace.



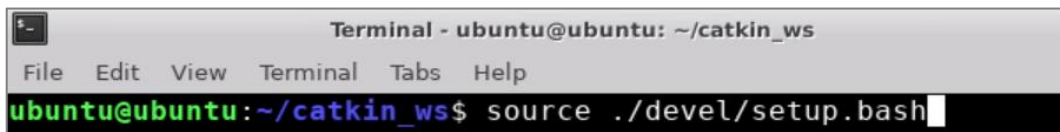
```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ cd ~/catkin_ws
```

- 2) Enter “catkin_make” command and press “Enter” to build all the packages in directory.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ catkin_make
```

- 3) Enter “source ./devel/setup.bash” command and press “Enter” to refresh the workspace environment.



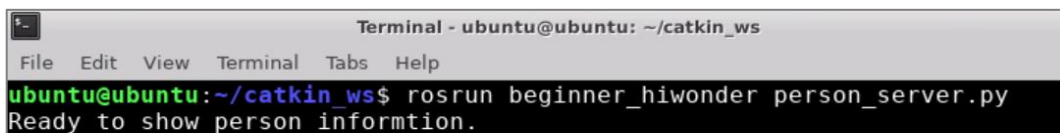
```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ source ./devel/setup.bash
```

- 4) Enter “roscore” command to start node manager.



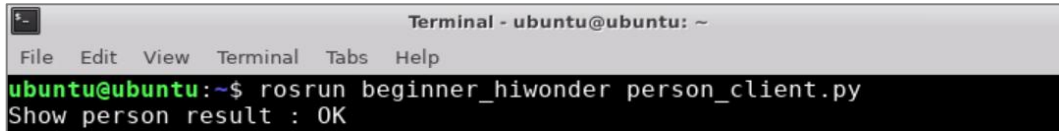
```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ roscore
```

- 5) Enter “roslaunch beginner_hiwonder person_publisher.py” command and press “Enter” to run publisher node. If want to stop running node, you can press “Ctrl+C”.



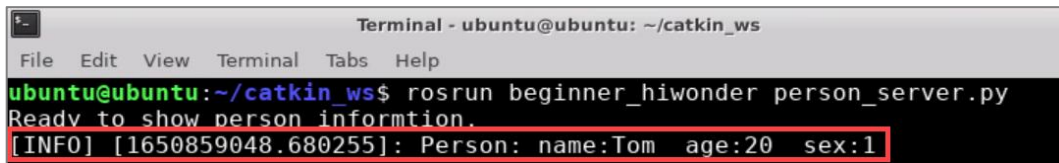
```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ roslaunch beginner_hiwonder person_publisher.py
Ready to show person information.
```

- 6) Open a new terminal. Enter “roslaunch beginner_hiwonder person_client.py” command and press “Enter” to run the client node.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosrun beginner_hiwonder person_client.py
Show person result : OK
```

- 7) After running the client node, the terminal window for starting server node will print the content shown in the red box in the figure below.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ rosrun beginner_hiwonder person_server.py
Ready to show person information.
[INFO] [1650859048.680255]: Person: name:Tom age:20 sex:1
```

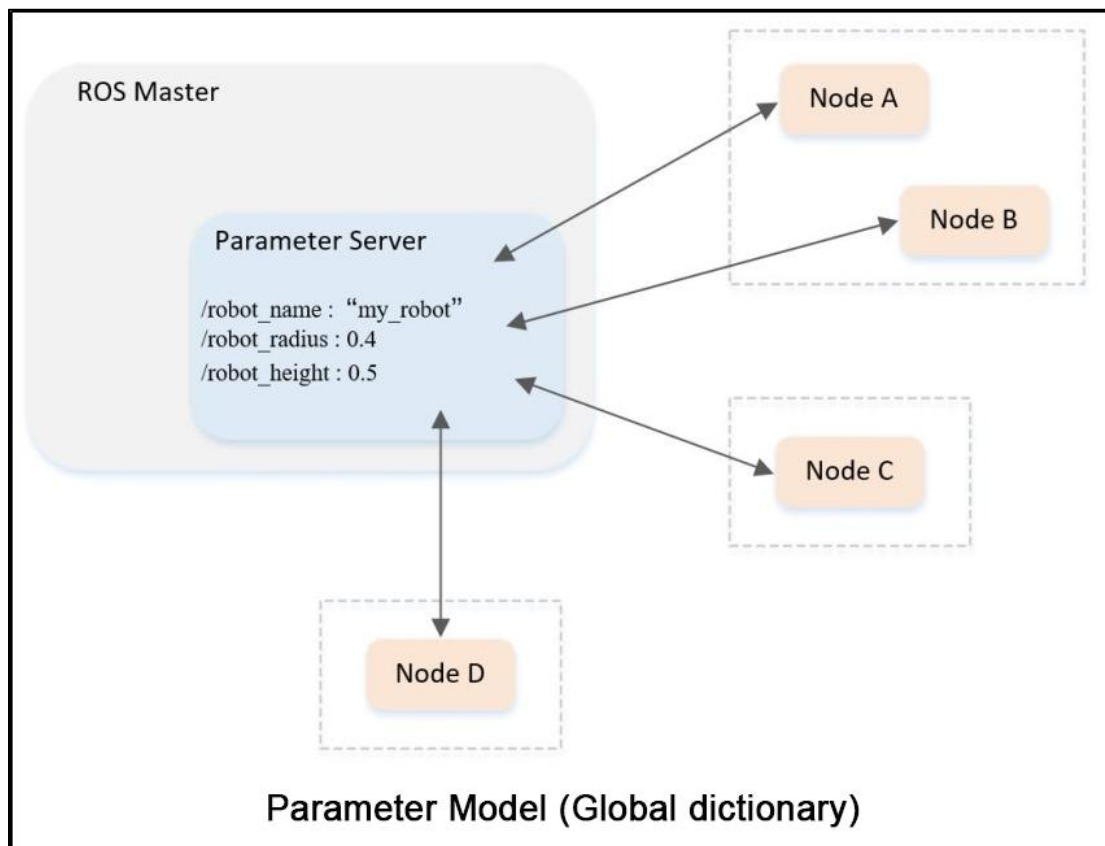
Lesson 11 Parameters Usage and Programming Method

1. Service Model

There is a parameter server in ROS master, which is a global dictionary to store configuration parameters among nodes. For example, parameter server for saving our name, radius and height can be globally accessible by each node.

If I access the robot name in Node A, I will get a value of “my_rot”. It only needs to send a query request to our ROS master, and then return the result of “my_rot”. The same goes for Node B, Node C, and Node D.

The parameter server model is shown in the following figure:



2. rosparam Parameter

2.1 rosparam Detailed Parameter

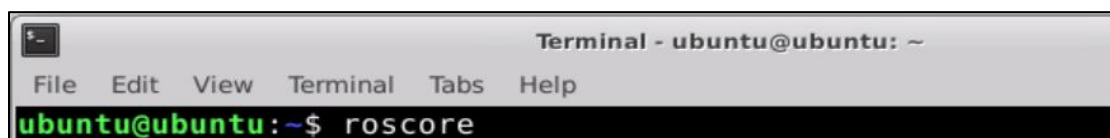
Let's get to know rosparam first and the detailed parameters are as follow:

- List the current number of parameter
`$ rosparam list`
- Display one of the parameter values
`$ rosparam get param_key`
- Set one of the parameter values
`$ rosparam set param_key param_value`
- Save the parameters to file
`$ rosparam dump file_name`
- Read the parameters from file
`$ rosparam load file_name`
- Delete parameter
`$ rosparam delete param_key`

2.2 Run Turtlesim Routine

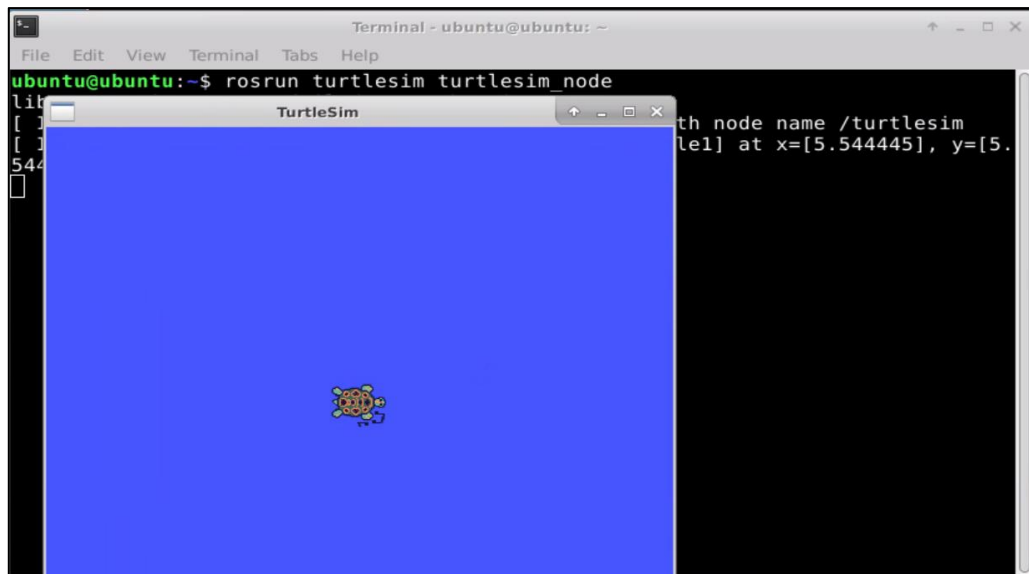
Taking turtlesim project as an example, run the turtlesim routine first. The specific operation steps are as follows:

- 1) Enter “roscore” command and press “Enter”.



Note: If the prompt “**roscore cannot run as another roscore/master is already running**” appears, which means node manager has been started before and this step can be skipped.

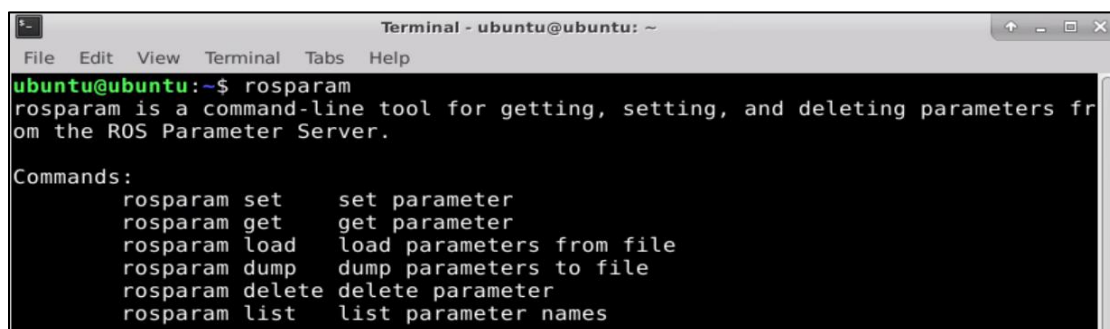
- 2) Enter the command “roslaunch turtlesim turtlesim_node”, and then press “Enter” to open the turtle simulator.



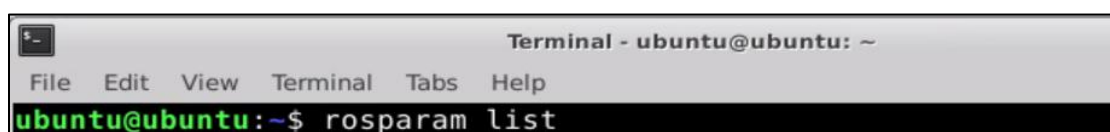
2.3 The Use of rosparam

The operation steps for the use of rosparam are as follow:

- 1) Open a new terminal.
- 2) Enter “rosparam” command and press “Enter”.

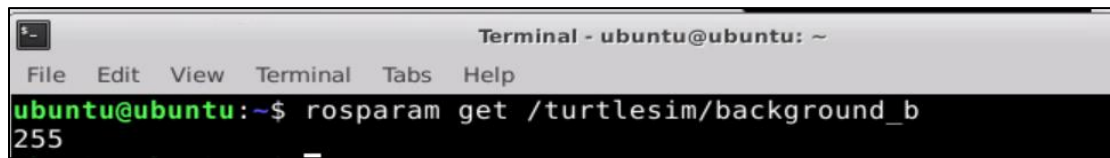


- 3) Enter the command "rosparam list" and press “Enter” to query the number of turtle parameters.



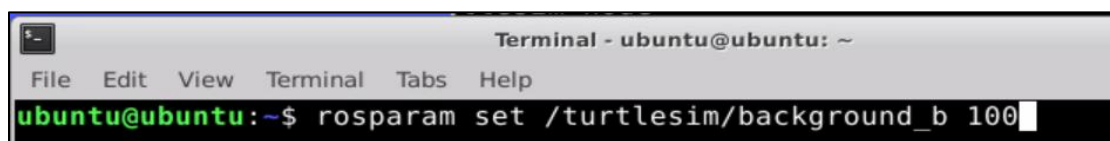
- 4) Enter “rosparam get /turtlesim/background_b” command and press “Enter”

to get the value of “background_b”. The same method goes for getting other values.



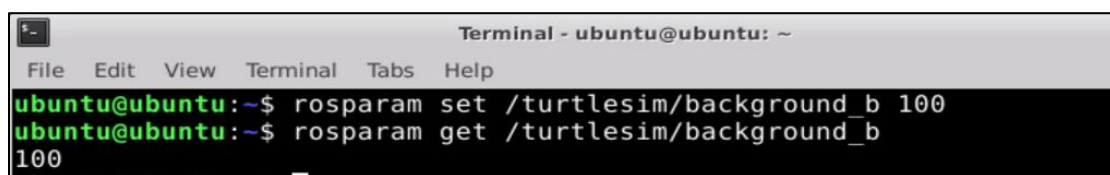
```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosparam get /turtlesim/background_b
255
```

5) Enter “rosparam set /turtlesim/background_b 100” command and press “Enter” to set “background_b” value. The same method goes for setting other values.



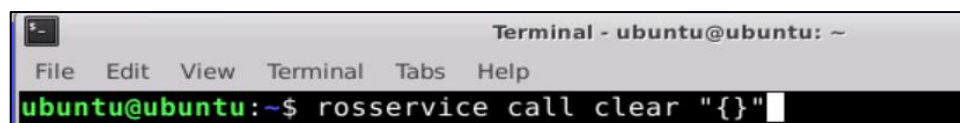
```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosparam set /turtlesim/background_b 100
```

6) Enter “rosparam get /turtlesim/background_b” command and press “Enter”. Then you can find that the value has been modified to 100.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosparam set /turtlesim/background_b 100
ubuntu@ubuntu:~$ rosparam get /turtlesim/background_b
100
```

7) Enter “rosservice call clear '{}'” command and press “Enter” to send the request to change color.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosservice call clear "{}"
```

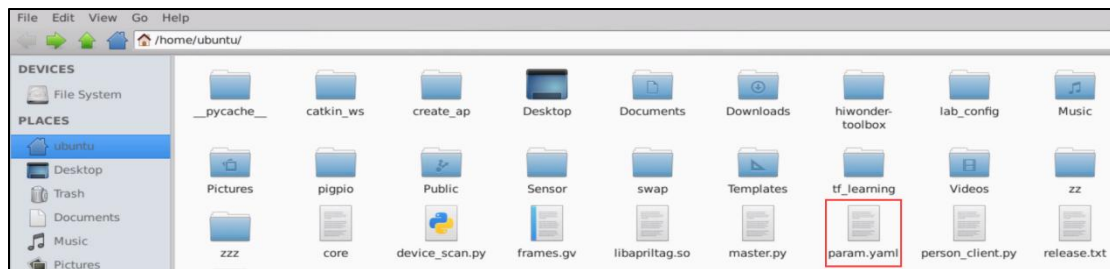


8) Enter “rosparam dump param.yaml” command and press “Enter” to save file.

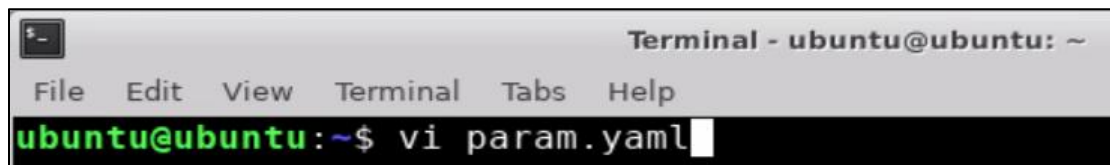


```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosparam dump param.yaml
```

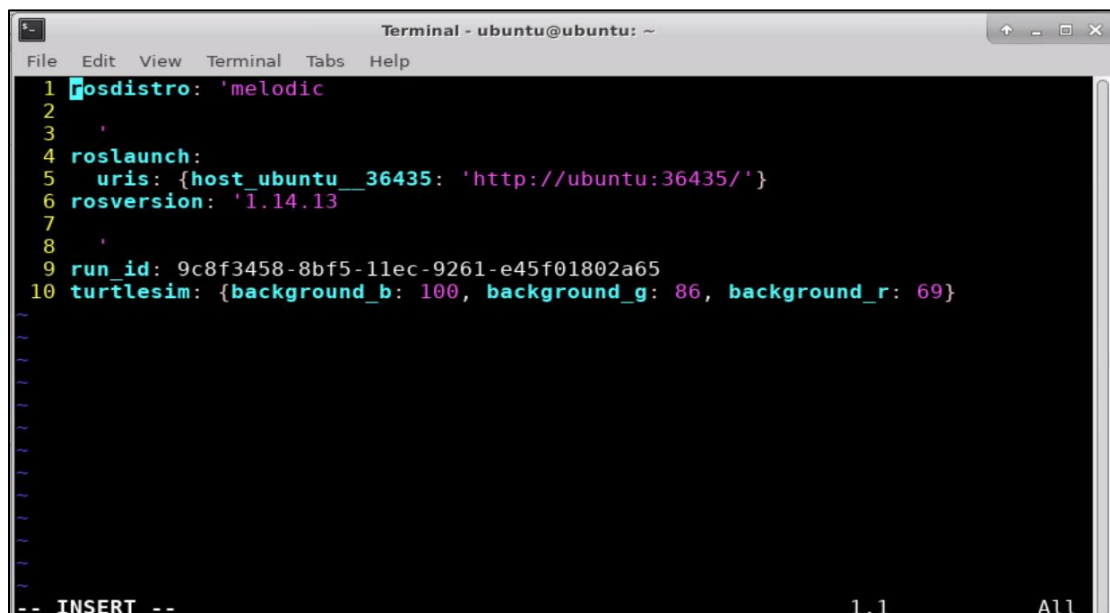
9) The created file is saved in the following path and open it directly.



10) Enter “vi param.yaml” command and press “Enter”. Then press “i” to modify “param.yaml”.

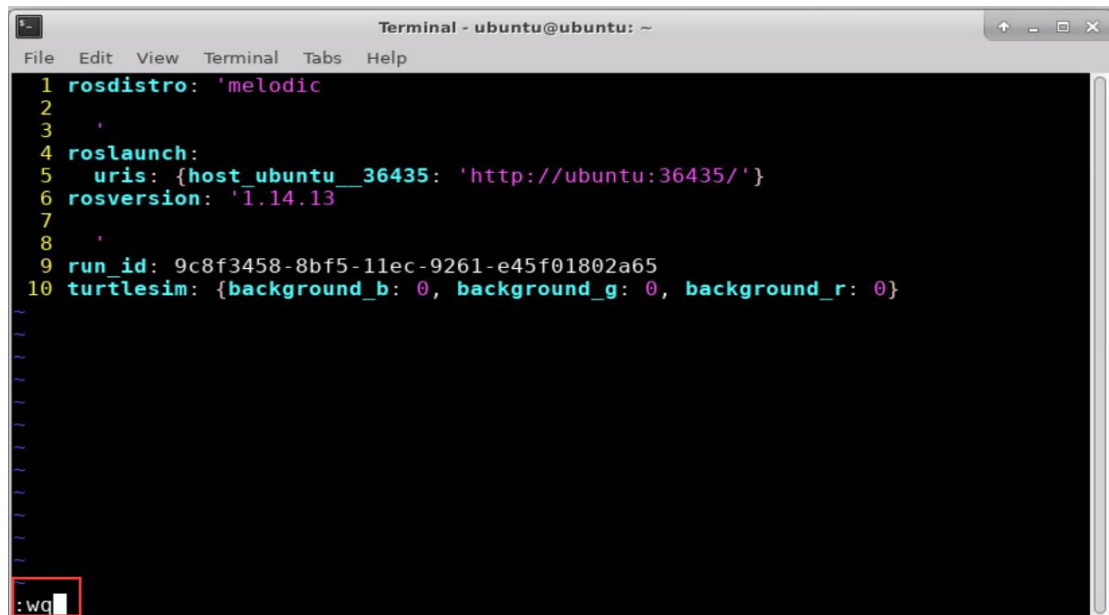


```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ vi param.yaml
```



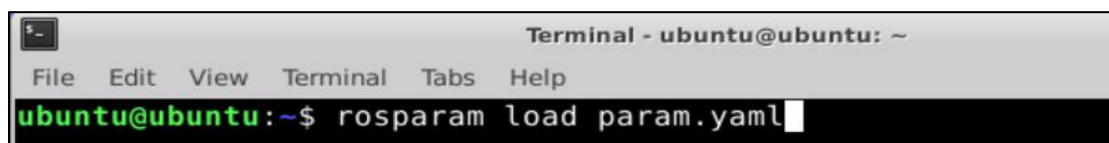
```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
1  Gosdistro: 'melodic'  
2  
3  
4  roslaunch:  
5    uris: {host_ubuntu_36435: 'http://ubuntu:36435/'}  
6  rosversion: '1.14.13'  
7  
8  
9  run_id: 9c8f3458-8bf5-11ec-9261-e45f01802a65  
10 turtlesim: {background_b: 100, background_g: 86, background_r: 69}  
  
-- INSERT --
```

11) The color can be modified to 0 which is black. After modifying, press “Esc” and enter “:wq” to save and exit.



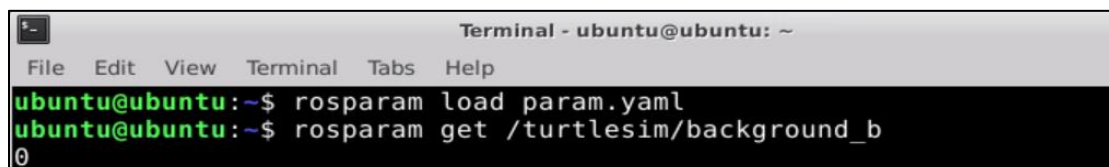
```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
1 rosdistro: 'melodic'
2
3
4 roslaunch:
5   uris: {host_ubuntu__36435: 'http://ubuntu:36435/'}
6   rosversion: '1.14.13'
7
8
9   run_id: 9c8f3458-8bf5-11ec-9261-e45f01802a65
10  turtlesim: {background_b: 0, background_g: 0, background_r: 0}
: wq
```

12) Enter “rosparam load param.yaml” command and press “Enter” to load the file.



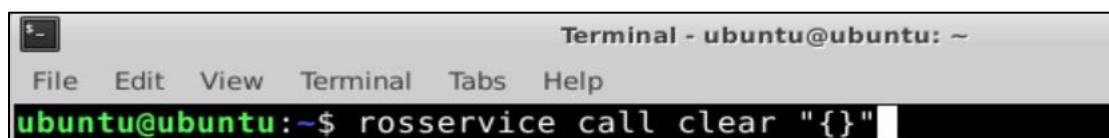
```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosparam load param.yaml
```

13) Enter “rosparam get /turtlesim/background_b” command and press “Enter” to check the loading effect.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosparam load param.yaml
ubuntu@ubuntu:~$ rosparam get /turtlesim/background_b
0
```

14) Enter “rosservice call clear '{}'” command and press “Enter” to send a request that changes the background scolor to black.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosservice call clear "{}"
```



15) Enter “rosparam delete /turtlesim/background_g” command and press “Enter” to delete the color of g.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosparam delete /turtlesim/background_g
```

16) Enter “rosparam list” command and press “Enter” to check effect.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosparam list
```

17) Enter “rosservice call clear '{}'” command and press “Enter” to refresh the background color to check effect.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosservice call clear "{}"
```



3. Programming Method

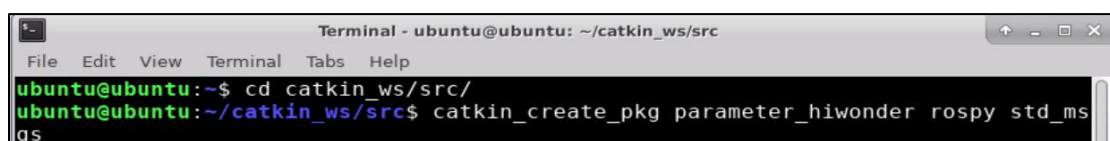
The operation steps for creating package is as follow:

- 1) Enter "cd catkin_ws/src/" command and press "Enter" to come to workspace.

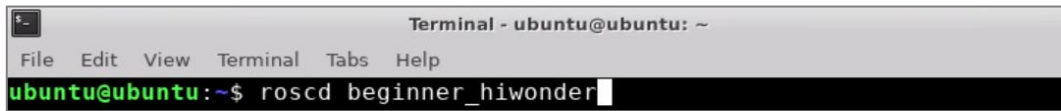


- 2) Enter "catkin_create_pkg parameter_hiwonder rospy std_msgs" command and press "Enter" to create package.

3.1 Write Control Program

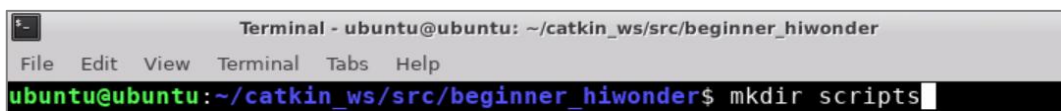


- 1) Open the terminal.
- 2) Enter “roscd parameter_hiwonder” command and press “Enter” to come to the package directory.



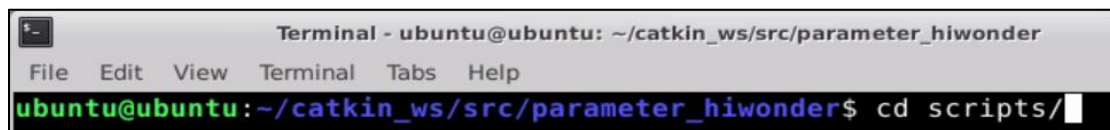
```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ roscd beginner_hiwonder
```

- 3) Enter “mkdir scripts” command and press “Enter” to create a new folder “scripts” where Python scripts are stored.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ mkdir scripts
```

- 4) Enter “cd scripts/” command and press “Enter” to come to the “scripts” folder.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/parameter_hiwonder  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/parameter_hiwonder$ cd scripts/
```

- 5) Enter the command “vi parameter_config.py” to edit the program, and then copy the following program. If want to modify, you can press “i”. After modifying, press “Esc” and enter “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/parameter_hiwonder/scripts  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/parameter_hiwonder/scripts$ vi parameter_config.py
```

```
#!/usr/bin/env python  
  
# -*- coding: utf-8 -*-  
  
# This routine set and read the parameter in turtle routine  
  
import sys
```

```
import rospy

from std_srvs.srv import Empty

def parameter_config():

    # Initialize ROS node

    rospy.init_node('parameter_config', anonymous=True)

    # read the parameter of blackground color

    red    = rospy.get_param('/turtlesim/background_r')

    green = rospy.get_param('/turtlesim/background_g')

    blue   = rospy.get_param('/turtlesim/background_b')

    rospy.loginfo("Get Backgroud Color[%d, %d, %d]", red, green, blue)

    # set the parameter of blackground color

    rospy.set_param("/turtlesim/background_r", 255);

    rospy.set_param("/turtlesim/background_g", 255);

    rospy.set_param("/turtlesim/background_b", 255);

    rospy.loginfo("Set Backgroud Color[255, 255, 255]");

    # read the parameter of blackground color
```

```
red    = rospy.get_param('/turtlesim/background_r')

green = rospy.get_param('/turtlesim/background_g')

blue   = rospy.get_param('/turtlesim/background_b')


rospy.loginfo("Get Backgroud Color[%d, %d, %d]", red, green, blue)


# After finding /spawn, create a service client, and then connect service named /spawn.

rospy.wait_for_service('/clear')

try:

    clear_background = rospy.ServiceProxy('/clear', Empty)

    # Request service call, enter request data

    response = clear_background()

    return response

except rospy.ServiceException, e:

    print "Service call failed: %s"%e


if __name__ == "__main__":

    parameter_config()
```

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/parameter_hiwonder/scripts
File Edit View Terminal Tabs Help
25
26     rospy.loginfo("Set Background Color[255, 255, 255]");
27
28     # 读取背景颜色参数
29     red = rospy.get_param('/turtlesim/background_r')
30     green = rospy.get_param('/turtlesim/background_g')
31     blue = rospy.get_param('/turtlesim/background_b')
32
33     rospy.loginfo("Get Background Color[%d, %d, %d]", red, green, blue)
34
35     # 发现 /spawn 服务后，创建一个服务客户端，连接名为 /spawn 的服务
36     rospy.wait_for_service('/clear')
37     try:
38         clear_background = rospy.ServiceProxy('/clear', Empty)
39
40         # 请求服务调用，输入请求数据
41         response = clear_background()
42         return response
43     except rospy.ServiceException, e:
44         print "Service call failed: %s"%e
45
46 if __name__ == "__main__":
47     parameter_config()
:wg

```

6) Enter “chmod +x parameter_config.py” command and press “Enter” to give the executable permission to the saved parameter_config.py.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws/src/parameter_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/parameter_hiwonder/scripts$ chmod +x parameter_config.py

```

4.2 Run Program

1) Enter “roscore” command and press “Enter” to start node manager.

```

Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ roscore

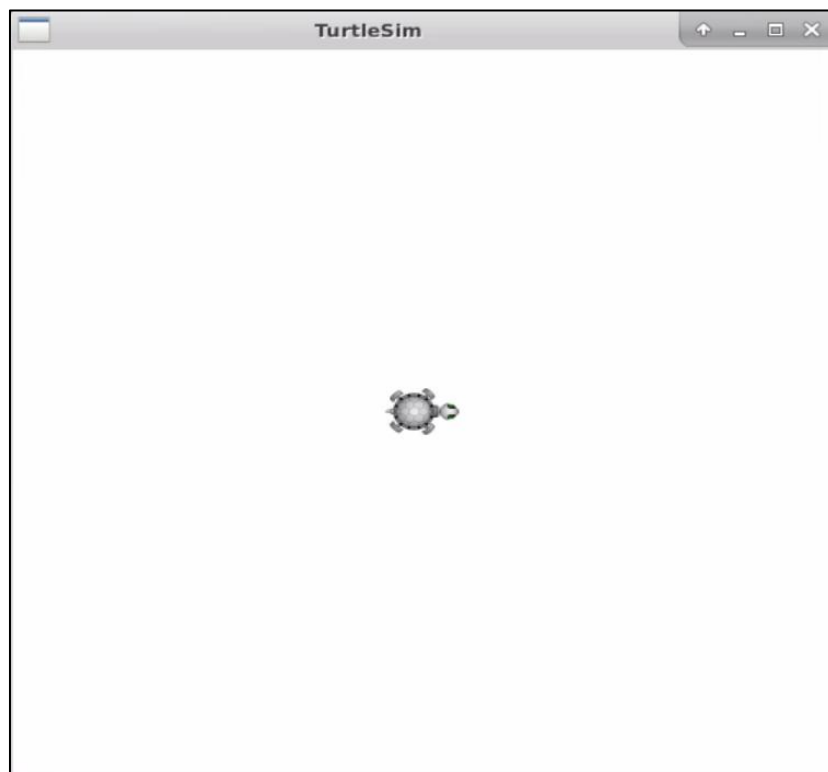
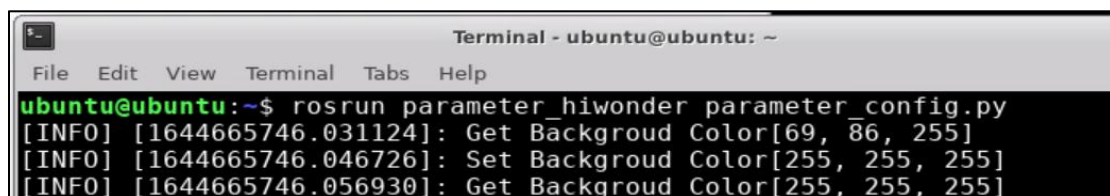
```

Note: If the prompt “roscore cannot run as another roscore/master is already running” appears, it means node manager has been started up so this step can be skipped directly.

2) Enter “roslaunch turtlesim turtlesim_node” command and press “Enter” to open the turtle simulator.



- 3) Enter “roslaunch parameter_hiwonder parameter_config.py” command and press “Enter” to run the program, which changes the background color of turtle to white, as the figure shown below:



- 4) If want to stop running program, you can press “Ctrl+C”.